

Program Analysis 2025-26

Lecture #2

Denotational Semantics



UNIVERSITÀ
DI PISA

Preliminaries

Semantics = assigning meaning to syntax

A program

c

syntax

(how the program is written)

IV

four

quattro

quatre

Its meaning

$\llbracket c \rrbracket$

semantics

(its computed function)

$\llbracket IV \rrbracket =$

$\llbracket \text{four} \rrbracket =$

$\llbracket \text{quattro} \rrbracket =$

$\llbracket \text{quatre} \rrbracket =$

4

What we say, what we mean

A boss to the employee

You showed great originality in your work

[[You showed great originality in your work]]

=

1. You improved the project with innovative ideas
2. You worked independently and confidently
3. You got everything wrong



What we say, what we mean

Politician to opponent

I appreciate your consistency in maintaining your position

[[I appreciate your consistency in maintaining your position]]

=

1. You are reliable and coherent in your ideas
2. You never change your mind even when proved wrong
3. You are loyal to your voters



What we say, what we mean

Landlord to lodger

I trust your well-known punctuality

[[I trust your well-known punctuality]]
=

1. I know you are always on time
2. I admire your disciplined lifestyle
3. Don't be late with your payment



What we say, what we mean

Between lovers

Ok, let's do whatever you think is best

[[Ok, let's do whatever you think is best]]

=

1. You'd better choose the option I like
2. I completely trust your judgment
3. I genuinely don't mind either option
4. I have no preference at all



What we say, what we mean

Public speech

I won't take long

[[I won't take long]]

=

1. This will only take a few minutes
2. I respect your time
3. I will summarize my points quickly
4. There are a lot of things I want to say



Rigorous approach vs jumbled words

Can Our Brains Really Read Jumbled Words as Long as The First And Last Letters Are Correct?

HUMANS 31 March 2018 By MICHELLE STARR

$$e^{i\pi} + 1 = 0$$

Accdrng to a rscheearch at Cmabrigde Uinervtisy, it deosn't mttar in waht oredr the ltteers in a wrod are, the olny iprmoetnt tihng is taht the frist and lsat ltteer be at the rghit pclae. The rset can be a toatl mses and you can sitll raed it wouthit porbelm. Tihs is bcuseae the huamn mnid deos not raed ervey lteter by istlef, but the wrod as a wlohe.

Some important sets

set of
integers

$\mathbb{Z}$

set of
natural numbers

$\mathbb{N}$

set of
boolean values

$\mathbb{B}$

state

set of
variables

$\sigma : X \rightarrow \mathbb{Z}$

set of
integers

set of all
states

$\Sigma \triangleq \{ \sigma : X \rightarrow \mathbb{Z} \}$

Notation: concrete states

$\sigma = [x \mapsto 2]$ the variable x has value 2,
all other variables have value 0

$\sigma = [x \mapsto 2, y \mapsto 3]$ the variable x has value 2,
the variable y has value 3,
all other variables have value 0

$\sigma = [y \mapsto 3, x \mapsto 2]$ same as above

$\sigma = [x \mapsto 2, x \mapsto 3]$ not a valid state!

Notation: state update

$$\sigma[n/y](x) \triangleq \begin{cases} n & \text{if } x = y \\ \sigma(x) & \text{otherwise} \end{cases}$$

$\sigma[2/x]$ the variable x has value 2,
any other variable y has value $\sigma(y)$

$\sigma[2/x][3/y]$ the variable x has value 2,
the variable y has value 3,
any other variable z has value $\sigma(z)$

$\sigma[3/y, 2/x]$ same as above

$\sigma[2/x][3/x]$ the variable x has value 3,
any other variable y has value $\sigma(y)$

What is the meaning of an arithmetic expression?

$$2x^2 + 3y + 5xz$$

$$[[2x^2 + 3y + 5xz]] : \Sigma \rightarrow \mathbb{Z}$$

=

1. a numerical value
2. a truth value
3. a state
4. a function

What is the meaning of a boolean expression?

$$2x^2 + 3y + 5xz \leq 25 \wedge x = y$$

$$\llbracket 2x^2 + 3y + 5xz \leq 25 \wedge x = y \rrbracket : \Sigma \rightarrow \mathbb{B}$$

=

1. a numerical value
2. a truth value
3. a state
4. a function

What is the meaning of a command?

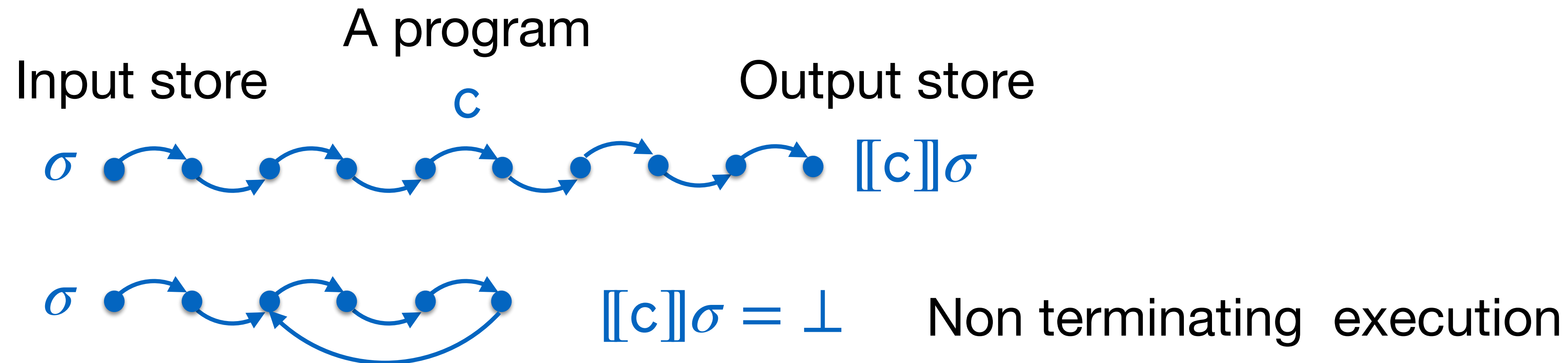
while $(x > 0)$ do $x := x - y$

$\llbracket \text{while } (x > 0) \text{ do } x := x - y \rrbracket : \Sigma \rightarrow \Sigma_{\perp}$
=

1. a numerical value
2. a truth value
3. a state
4. a function

Semantics of deterministic programs

Given the input state we want to define the **reachable output states**

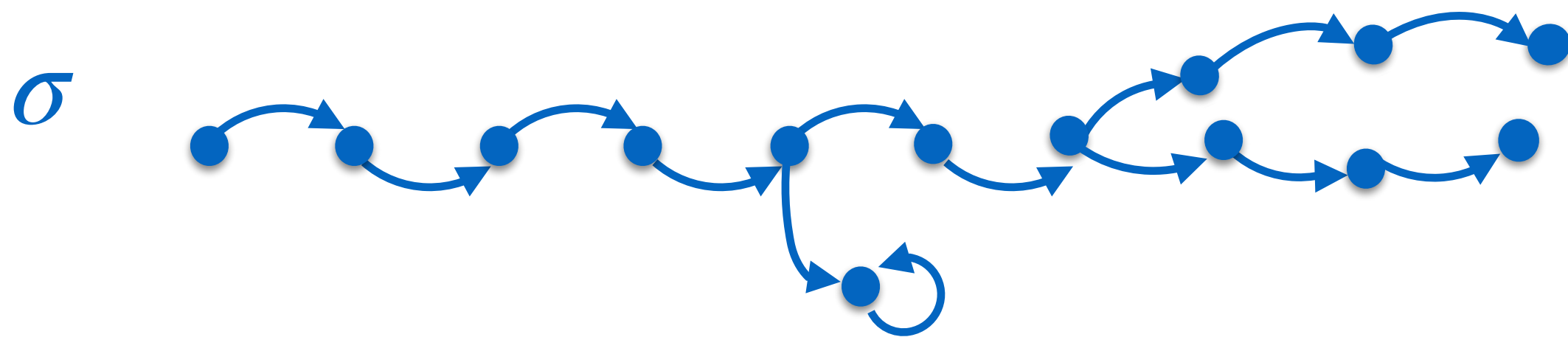


Denotational semantics $[[c]] : \Sigma \rightarrow \Sigma_{\perp}$ $\Sigma_{\perp} = \Sigma \uplus \{ \perp \}$

Nondeterministic programs

Some programs may exhibit nondeterministic behaviour
(lack of information, approximation, programming constructs $c_1 + c_2$)

A program c

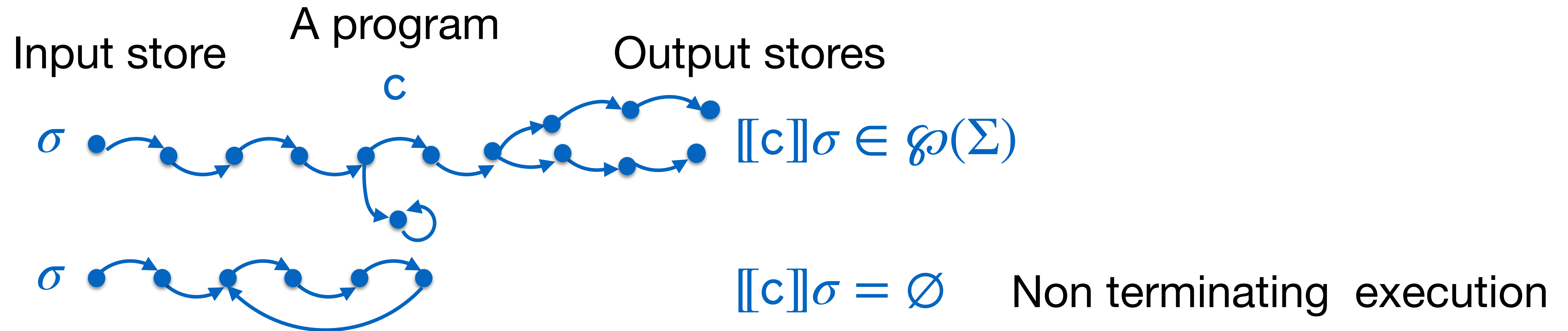


from $\llbracket c \rrbracket : \Sigma \rightarrow \Sigma_{\perp}$

to $\llbracket c \rrbracket : \Sigma \rightarrow \wp(\Sigma)$

Collecting semantics

Given the input state we want to define the **reachable output states**



Collecting semantics

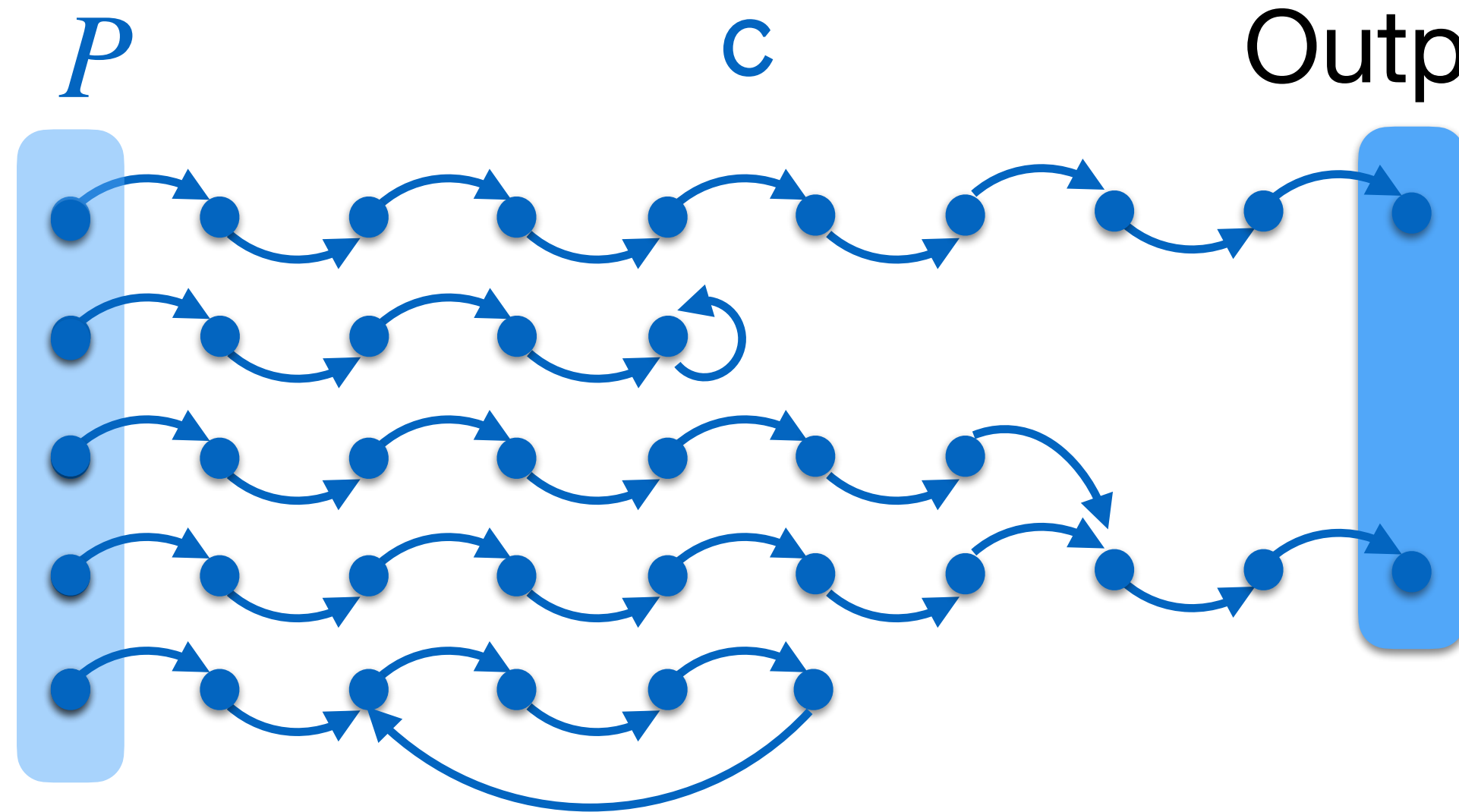
$$[[c]] : \wp(\Sigma) \rightarrow \wp(\Sigma)$$

Collecting semantics

Input stores

A program

Output stores



We write $\llbracket c \rrbracket \sigma$
as a shorthand for $\llbracket c \rrbracket \{\sigma\}$

$$\llbracket c \rrbracket P = \bigcup_{\sigma \in P} \llbracket c \rrbracket \sigma$$

Collecting semantics

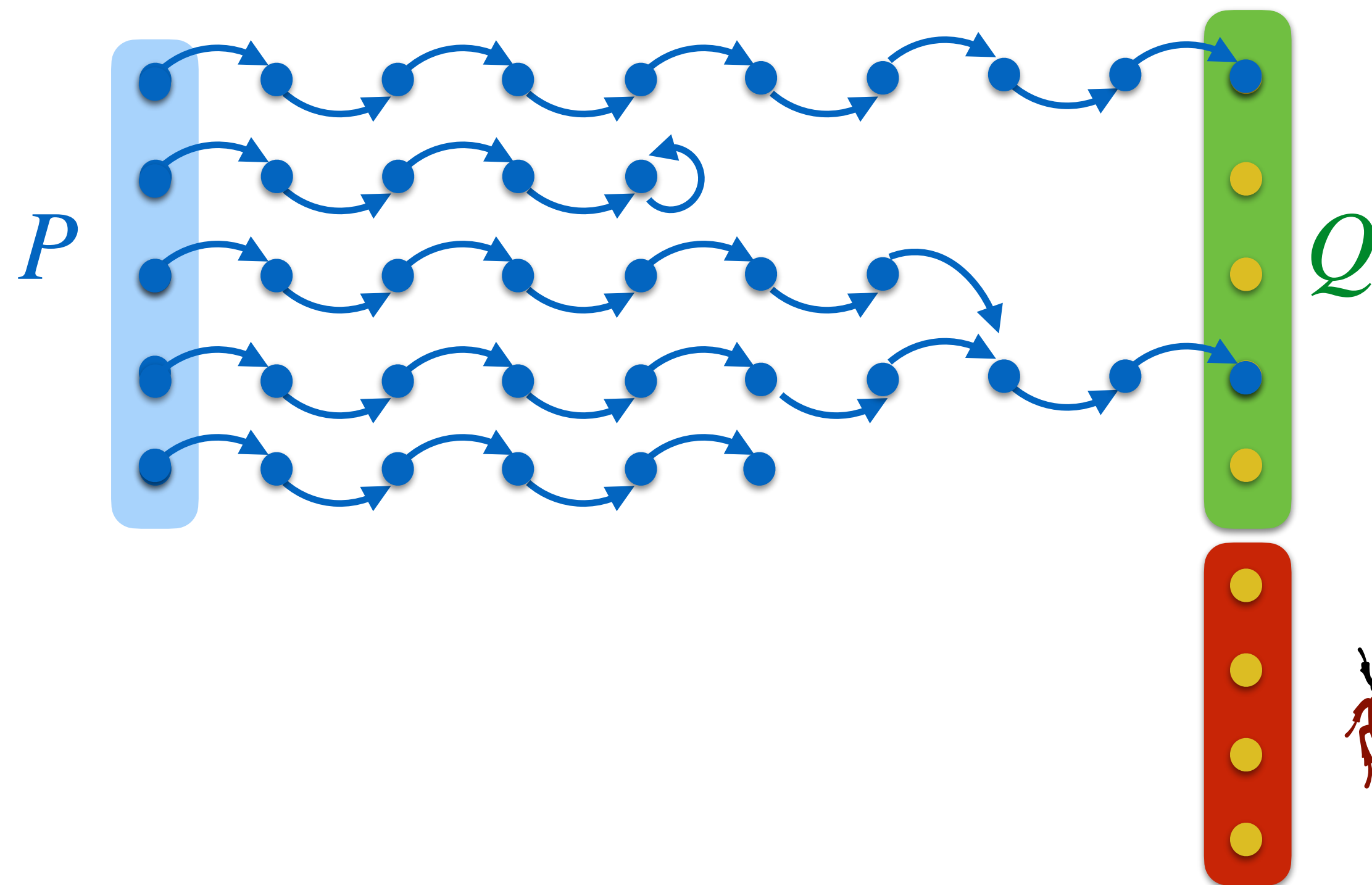
$\llbracket c \rrbracket : \wp(\Sigma) \rightarrow \wp(\Sigma)$ Consider all trajectories simultaneously

Partial correctness

A program

c

$$[[c]]P \subseteq Q$$



$\forall \sigma \in P . [[c]]\sigma$

- either does not terminate
- or terminates in Q

No undesirable state is reached!

Dijkstra's weakest liberal precondition

Given a program c and a correctness specification Q
what is the largest set of "safe" input states that we can start from?

$$wlp(c, Q) = \{\sigma \mid \llbracket c \rrbracket \{\sigma\} \subseteq Q\}$$

$wlp(c, Q)$ is the set of all and only partially correct states

$$P \subseteq wlp(c, Q)$$

if and only if

$$\llbracket c \rrbracket P \subseteq Q$$

Dijkstra's weakest liberal precondition

Given a program c and a correctness specification Q
what is the largest set of "safe" input states that we can start from?

$$wlp(c, Q) = \{\sigma \mid \llbracket c \rrbracket \{\sigma\} \subseteq Q\}$$

$wlp(c, Q)$ is the set of all and only partially correct states

$$P \subseteq wlp(c, Q)$$

if and only if

$$\llbracket c \rrbracket P \subseteq Q$$

$$\llbracket x = x + 1 \rrbracket (x+1 > 30) \subseteq (x > 0)$$

$$\llbracket x = x + 1 \rrbracket (x+1 > 0) \subseteq (x > 0)$$

$$\{x \mid x + 1 > 30\} \subseteq \{x \mid x + 1 > 0\}$$

Dijkstra's weakest liberal precondition

Given a program c and a correctness specification Q
what is the largest set of "safe" input states that we can start from?

$$wlp(c, Q) = \{\sigma \mid \llbracket c \rrbracket \{\sigma\} \subseteq Q\}$$

$wlp(c, Q)$ is the set of all and only partially correct states

$$P \subseteq wlp(c, Q)$$

if and only if

$$\llbracket c \rrbracket P \subseteq Q$$

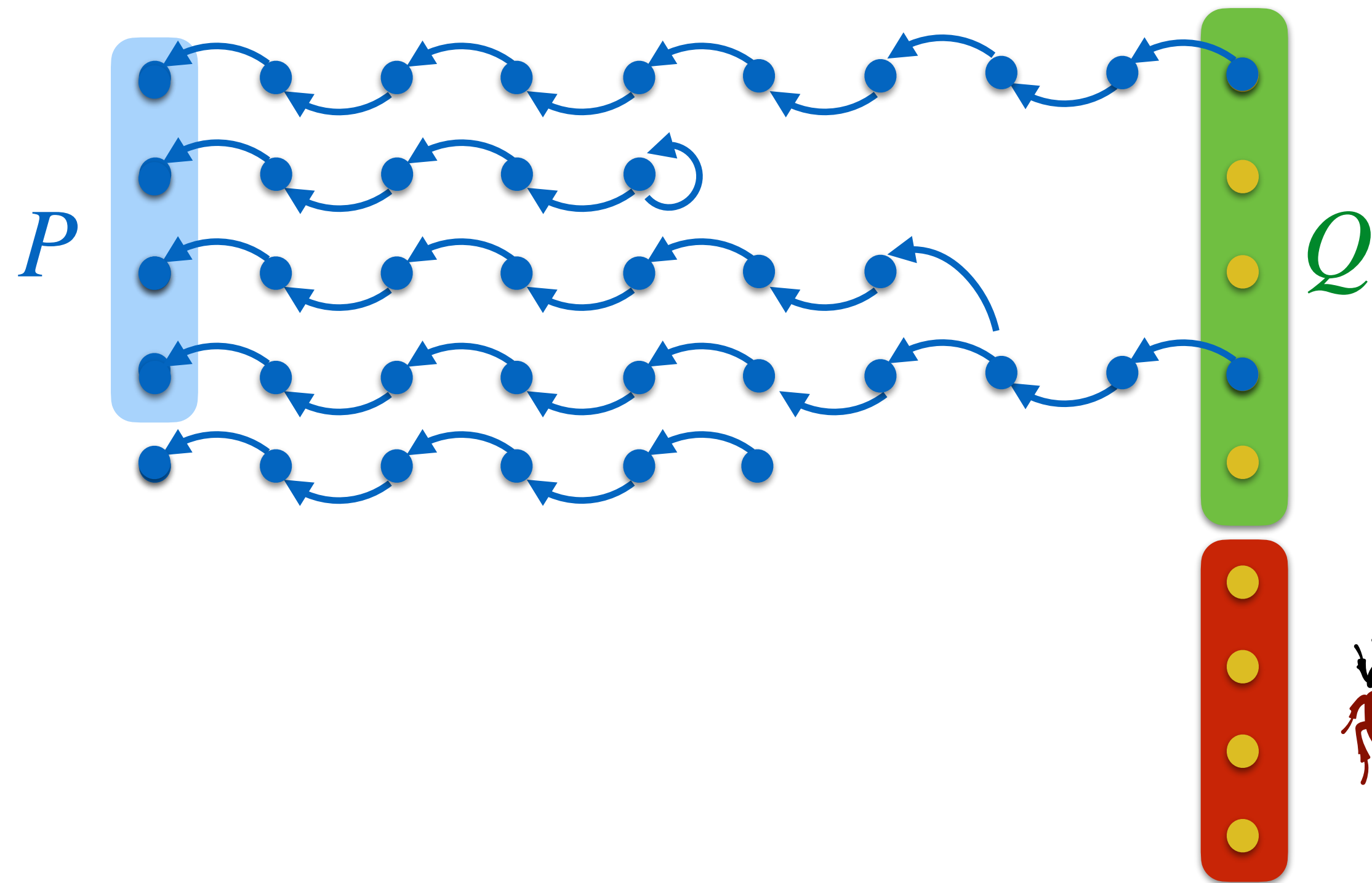
$$\llbracket c \rrbracket (wlp(c, Q)) \stackrel{?}{\subseteq} Q$$

$$wlp(c, \llbracket c \rrbracket P) \stackrel{?}{\supseteq} P$$

Partial correctness: backward

A program

c



$$P \subseteq wlp(c, Q)$$

$$\llbracket c \rrbracket P \subseteq Q$$

Admissible outputs
described in a
correctness specification

Dijkstra's weakest liberal precondition

$$wlp(c, Q) = \{\sigma \mid \llbracket c \rrbracket \{\sigma\} \subseteq Q\}$$

Predicates vs sets

$$(x = 1)$$

$$\{\sigma \mid \sigma(x) = 1\}$$

$$(x \leq y)$$

$$\{\sigma \mid \sigma(x) \leq \sigma(y)\}$$

$$P$$

$$\{\sigma \models P\} = \{\sigma \mid \llbracket P \rrbracket \sigma = \text{true}\}$$

Example

$c \triangleq$

```
% let us focus on positive numbers
% non-trivial divisors of x
s := nondet[2..x/2];
if (x%s=0)
    skip
else
    while true do skip
```

$$\llbracket c \rrbracket(x = 1) = \emptyset$$

$$\llbracket c \rrbracket(x = 35) = (x = 35 \wedge s \in \{5, 7\})$$

$$\llbracket c \rrbracket(x = 12) = (x = 12 \wedge s \in \{2, 3, 4, 6\})$$

$$\begin{aligned} wlp(c, (s \in \{2, 3\})) &= (x \in \{1, 4, 6, 9\}) \\ &\quad \cup (x \text{ prime}) \end{aligned}$$

Exactness vs approximation

$c \triangleq$

```
% let us focus on positive numbers
```

```
% non-trivial divisors of x
```

```
s := nondet[1..];
```

```
while s>1 do {
```

```
    x := 2*x;
```

```
    s := s-1
```

```
}
```

$$\llbracket c \rrbracket(x = 1) = (s = 1 \wedge \exists n. x = 2^n)$$

$$\llbracket c \rrbracket(x = 1) \subseteq (\exists n. x = 2^n)$$

$$\llbracket c \rrbracket(x = 1) \subseteq (x \geq 1)$$

$$\llbracket c \rrbracket(x = 1) \subseteq (s \geq 1 \wedge x \geq 1)$$

over-approximations

Exactness vs approximation

$c \triangleq$

```
% let us focus on positive numbers
% non-trivial divisors of x
s := nondet[1..];
while s>1 do {
    x := 2*x;
    s := s-1
}
```

$$[[c]](x = 1) = (s = 1 \wedge \exists n . x = 2^n)$$

$$[[c]](x = 1) \supseteq (s = 1 \wedge x \in \{2,4,8,16\})$$

$$[[c]](x = 1) \supseteq (s = 1 \wedge \exists n . x = 8^n)$$

$$[[c]](x = 1) \supseteq (s = 1 \wedge x = 256)$$

under-approximations

Approximations?

$c \triangleq$

```
% let us focus on positive numbers
% non-trivial divisors of x
s := nondet[1..];
while s>1 do {
    x := 2*x;
    s := s-1
}
```

$$\llbracket c \rrbracket(x = 1) = (s = 1 \wedge \exists n. x = 2^n)$$

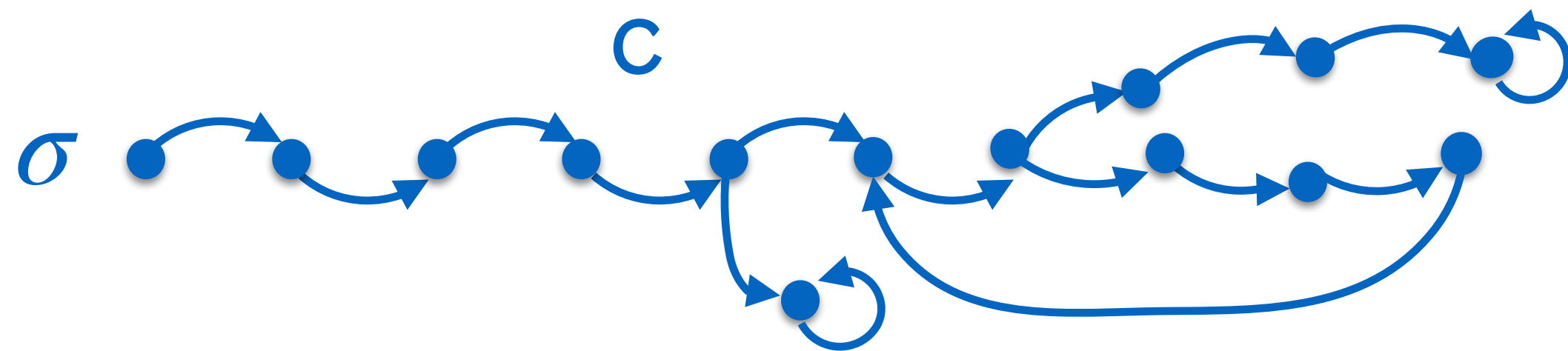
$$\llbracket c \rrbracket(x = 1) \not\equiv ? (x \in \{2, 4, 8, 16\})$$

$$\llbracket c \rrbracket(x = 1) \subseteq ? (s \leq x)$$

$$\llbracket c \rrbracket(x = 1) \supseteq ? (s = 0 \wedge x = 2 \wedge y = x)$$

Non-termination analysis

Given a program c and an input store σ does $\llbracket c \rrbracket \sigma = \emptyset$?



Non
termination

- Find an over-approximation Q of $\llbracket c \rrbracket \sigma$ such that $Q \subseteq \emptyset$

- Find an under-approximation Q of $\llbracket c \rrbracket \sigma$ such that $Q \supset \emptyset$

Termination

Mathematics

Disclaimer

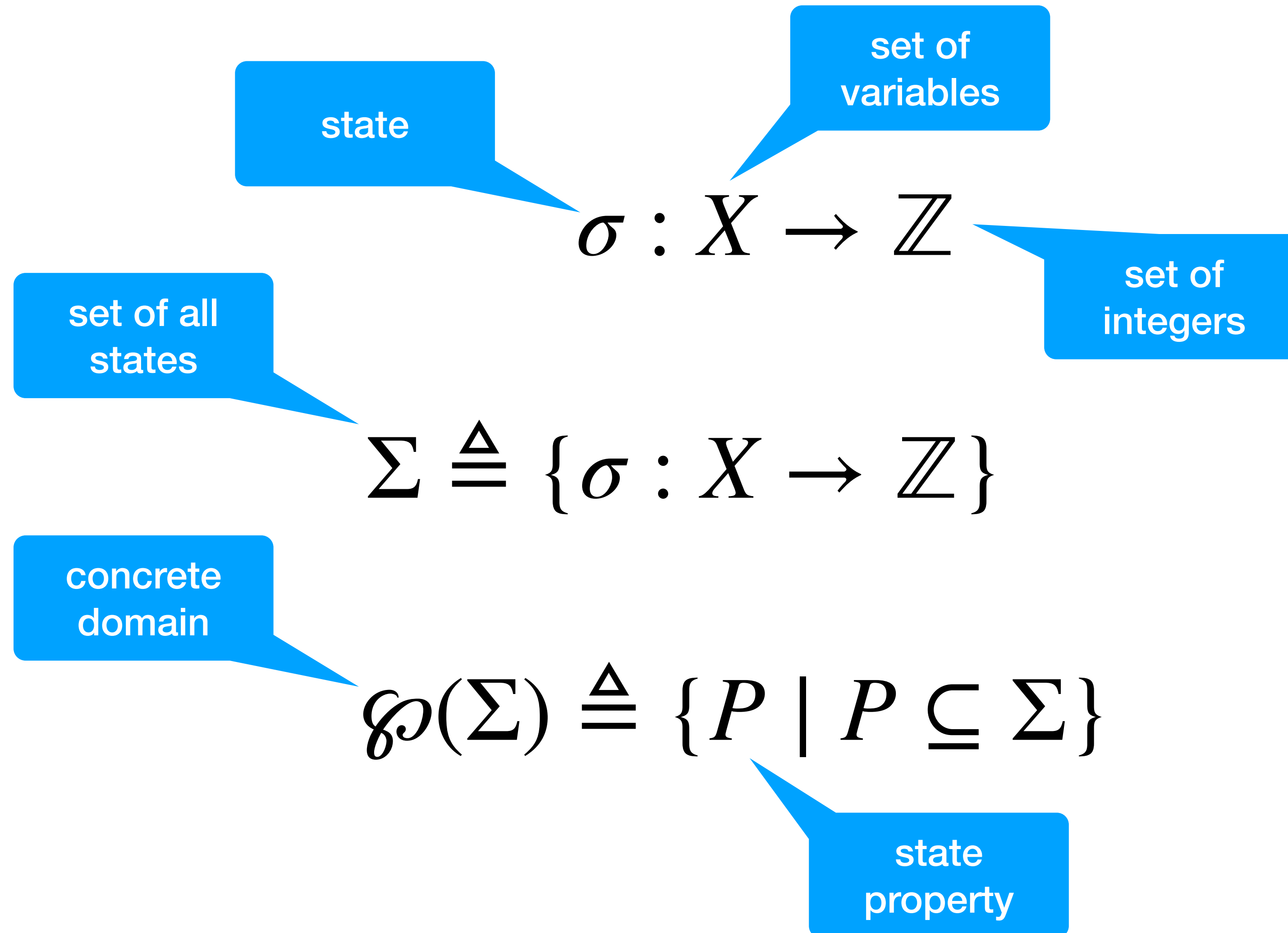


This part of the presentation was mostly prepared late at night

While every effort has been made to ensure accuracy, minor errors or oversights may be present

It is your task to catch them

Concrete domain



Notation

$[x \mapsto 1, y \mapsto 2]$

state
notation

the state where x holds 1, y holds 2 and any other variable holds 0

$\sigma[n/x]$

state
update

$\sigma[x \mapsto n]$

sometimes
written

the state where x holds n and any other variable y holds $\sigma(y)$

$(x = 1, y = 2)$

conjunction

property
notation

the set of all states where x holds 1 **and** y holds 2

Regular commands

regular
command

$c ::=$

e

|

$c_1; c_2$

|

$c_1 + c_2$

|

$c^\star$

atomic
command

choice

Kleene
star

$e ::= \text{skip} \mid x := a \mid b? \mid \dots$

arithmetic
expression

$a ::= n \mid x \mid a_1 + a_2 \mid \dots$

boolean
expression

$b ::= a_1 \leq a_2 \mid b_1 \wedge b_2 \mid \dots$

Arithmetic expressions



arithmetic
expression

$$a ::= n \mid x \mid a_1 + a_2 \mid \dots$$

Arithmetic expressions

$$[[\cdot]] : Aexp \rightarrow \Sigma \rightarrow \mathbb{Z}$$

$$[[a]]\sigma$$

evaluates the arithmetic expression a in the current state σ

$$[[n]]\sigma \triangleq n$$

$$[[x]]\sigma \triangleq \sigma(x)$$

$$[[a_0 \text{ op } a_1]]\sigma \triangleq [[a_0]]\sigma \text{ op } [[a_1]]\sigma$$

syntactic
operator

mathematical
operation

Example

$$\llbracket 2x^2 + 3y + 5xz \rrbracket : \Sigma \rightarrow \mathbb{Z}$$

$$\llbracket 2x^2 + 3y + 5xz \rrbracket [x \mapsto 1, y \mapsto 2, z \mapsto 1] = 13$$

$$\llbracket 2x^2 + 3y + 5xz \rrbracket [x \mapsto 1, z \mapsto 1] = 7$$

$$\llbracket 2x^2 + 3y + 5xz \rrbracket [x \mapsto 1] = 2$$

Boolean expressions



boolean
expression

$$b ::= a_1 \leq a_2 \mid b_1 \wedge b_2 \mid \dots$$

Boolean expressions

collecting
version

$$[[\cdot]]: \text{Bexp} \rightarrow \wp(\Sigma) \rightarrow \wp(\Sigma)$$

$[[b]]P$ (intuitively $b \wedge P$)

is the set of all and only states in P that satisfy the condition b

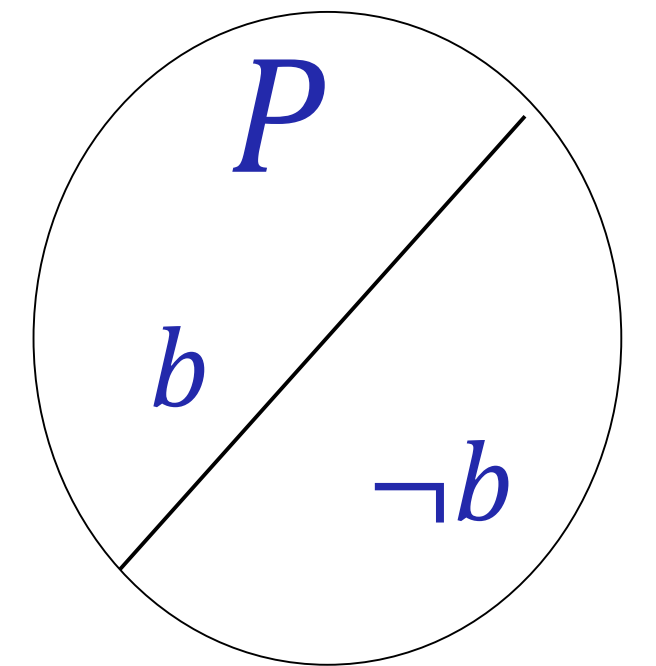
$$[[\text{true}]]P \triangleq P$$

$$[[\text{false}]]P \triangleq \emptyset$$

$$[[a_0 \text{ cmp } a_1]]P \triangleq \{ \sigma \in P \mid [[a_0]]\sigma \text{ cmp } [[a_1]]\sigma \}$$

$$[[b_0 \text{ bop } b_1]]P \triangleq [[b_0]]P \text{ bop } [[b_1]]P$$

$$[[\text{not } b]]P \triangleq P \setminus ([[b]]P)$$



Lemma
 $[[b]]P \subseteq P$

Example

$$\llbracket 2x^2 + 3y + 5xz \leq 25 \wedge x = y \rrbracket : \wp(\Sigma) \rightarrow \wp(\Sigma)$$

$$\llbracket 2x^2 + 3y + 5xz \leq 25 \wedge x = y \rrbracket (x = 1) = (x = y = 1, z \leq 4)$$

$$\llbracket 2x^2 + 3y + 5xz \leq 25 \wedge x = y \rrbracket (y = 2) = (x = y = 2, z \leq 1)$$

$$\llbracket 2x^2 + 3y + 5xz \leq 25 \wedge x = y \rrbracket (x, z \geq 3) = \text{false}$$

Assertion language

assertion

$P ::= \text{true} \mid \text{false} \mid a_1 < a_2 \mid a_1 = a_2 \mid \dots$
 $\mid \neg P \mid P_1 \wedge P_2 \mid \exists x. P \mid \dots$

Boolean and
classical
assertions

$\sigma \models P$ or also $\sigma \in P$

the state σ satisfies the property P

$P \Rightarrow Q$ or also $P \subseteq Q$ or also $P \leq Q$

any state that satisfies P satisfies Q

Regular commands

regular
command

atomic
command

$e ::= \text{skip} \mid x := a \mid b? \mid \dots$

$c ::=$

e
 $\mid$
 $c_1; c_2$

choice

$\mid$
 $c_1 + c_2$

$\mid$
 $c^\star$

Kleene
star

Kleene star, informally

$c^\star = \text{skip}$

+ c

+ $c;c$

+ $c;c;c$

+ ...

+ $c;...;c$ (n times, also written c^n)

+ ...

Collecting semantics

concrete
semantics

$$[[c]] : \wp(\Sigma) \rightarrow \wp(\Sigma)$$

$$[[c]]P$$

is the set of all and only states reachable from some state in P after executing c

$$[[c]]\sigma \text{ as a shorthand for } [[c]]\{\sigma\}$$

$$\text{additivity: } [[c]](P_1 \cup P_2) = ([[c]]P_1) \cup ([[c]]P_2)$$

Atomic commands

$$[[\cdot]] : \text{Com} \rightarrow \wp(\Sigma) \rightarrow \wp(\Sigma)$$

$$[[\text{skip}]]P \triangleq P$$

$$[[x := a]]P \triangleq \{ \sigma[n/x] \mid \sigma \in P, n = [[a]]\sigma \}$$

$$[[b?]]P \triangleq [[b]]P$$

Regular commands

$$\llbracket \cdot \rrbracket : \text{Com} \rightarrow \wp(\Sigma) \rightarrow \wp(\Sigma)$$

$$\llbracket c_1; c_2 \rrbracket P \triangleq \llbracket c_2 \rrbracket (\llbracket c_1 \rrbracket P)$$

$$\llbracket c_1 + c_2 \rrbracket P \triangleq \llbracket c_1 \rrbracket P \cup \llbracket c_2 \rrbracket P$$

$$\llbracket c^\star \rrbracket P \triangleq \bigcup_{k=0}^{\infty} \llbracket c \rrbracket^k P$$

$$f : D \rightarrow D$$

$$f^0(d) = d$$

$$f^{n+1}(d) = f(f^n(d))$$

How to compute loops?

$$\begin{aligned} \llbracket c^\star \rrbracket P &\triangleq \bigcup_{k=0}^{\infty} \llbracket c \rrbracket^k P = \llbracket c \rrbracket^0 P \cup \bigcup_{k=1}^{\infty} \llbracket c \rrbracket^k P \\ &= P \cup \bigcup_{k=0}^{\infty} \llbracket c \rrbracket^{k+1} P = P \cup \llbracket c \rrbracket \bigcup_{k=0}^{\infty} \llbracket c \rrbracket^k P \\ &= P \cup \llbracket c \rrbracket (\llbracket c^\star \rrbracket P) \\ &= \text{fix } \lambda X. (P \cup \llbracket c \rrbracket X) \end{aligned}$$

Let's use Kleene's fixpoint theorem!
(from MPP, 1st semester)

We look for the **smallest** set of states that

- **contains** P and
- is "**closed**" under $\llbracket c \rrbracket$

Encoding conditionals

$$\text{if } b \text{ then } c_1 \text{ else } c_2 \triangleq (b?; c_1) + (\neg b?; c_2)$$

$$\llbracket \text{if } b \text{ then } c_1 \text{ else } c_2 \rrbracket P \triangleq \llbracket c_1 \rrbracket (\llbracket b \rrbracket P) \cup \llbracket c_2 \rrbracket (\llbracket \neg b \rrbracket P)$$

e.g.

$$\begin{aligned} & \llbracket \text{if } x \geq 0 \text{ then skip else } x := -x \rrbracket (x \in \{-1, 1\}) \\ & \triangleq \llbracket \text{skip} \rrbracket (x = 1) \cup \llbracket x := -x \rrbracket (x = -1) \\ & \triangleq (x = 1) \end{aligned}$$

Encoding while commands

$$\text{while } b \text{ do } c \triangleq (b?; c)^{\star}; \neg b?$$

$$\llbracket \text{while } b \text{ do } c \rrbracket P \triangleq \llbracket \neg b \rrbracket \bigcup_{k=0}^{\infty} (\llbracket c \rrbracket \circ \llbracket b \rrbracket)^k P$$

Example

$$\llbracket \text{while } b \text{ do } c \rrbracket P \triangleq \llbracket \neg b \rrbracket \bigcup_{k=0}^{\infty} (\llbracket c \rrbracket \circ \llbracket b \rrbracket)^k P$$

e.g.

$w \triangleq \text{while } y \leq r \text{ do}$

$r := r - y;$

$q := q + 1$

$\sigma \triangleq [x \mapsto 5,$
 $y \mapsto 2,$
 $r \mapsto 5]$

$$f \triangleq \llbracket r := r - y; q := q + 1 \rrbracket \circ \llbracket y \leq r \rrbracket$$

$$P_0 = \{\sigma\}$$

$$P_{k+1} \triangleq \{\sigma\} \cup f(P_k)$$

$$P_0 = \{\sigma\}$$

$$P_1 = \{\sigma\} \cup f(P_0) = \{\sigma, [x \mapsto 5, y \mapsto 2, r \mapsto 3, q \mapsto 1]\}$$

$$P_2 = \{\sigma\} \cup f(P_1) =$$

$$\{\sigma, [x \mapsto 5, y \mapsto 2, r \mapsto 3, q \mapsto 1], [x \mapsto 5, y \mapsto 2, r \mapsto 1, q \mapsto 2]\}$$

$$P_3 = \{\sigma\} \cup f(P_2) = P_2 \text{ we can stop!}$$

$$\llbracket w \rrbracket \{\sigma\} = \llbracket y > r \rrbracket P_3 = \{[x \mapsto 5, y \mapsto 2, r \mapsto 1, q \mapsto 2]\}$$

Relational semantics

Collecting semantics

$$\llbracket c \rrbracket : \wp(\Sigma) \rightarrow \wp(\Sigma)$$

$\llbracket c \rrbracket$ can be equivalently seen as the relation $\llbracket c \rrbracket \subseteq \Sigma \times \Sigma$

$$\llbracket c \rrbracket = \{(\sigma, \delta) \mid \sigma \in \Sigma, \delta \in \llbracket c \rrbracket \sigma\}$$

Regular commands

$$[\text{skip}] \triangleq \{ (\sigma, \sigma) \mid \sigma \in \Sigma \}$$

$$[x := a] \triangleq \{ (\sigma, \sigma[[a]]\sigma/x) \mid \sigma \in \Sigma \}$$

$$[b?] \triangleq \{ (\sigma, \sigma) \mid \sigma \models b \}$$

$$[c_1; c_2] \triangleq [c_2] \circ [c_1]$$

$$\triangleq \{ (\sigma, \delta) \mid \exists \gamma \in \Sigma, (\sigma, \gamma) \in [c_1], (\gamma, \delta) \in [c_2] \}$$

$$[c_1 + c_2] \triangleq [c_1] \cup [c_2]$$

$$[c^\star] \triangleq \bigcup_{k=0}^{\infty} [c]^k$$

Backward semantics

$\llbracket c \rrbracket^{\text{op}} \triangleq \{ (\delta, \sigma) \mid (\sigma, \delta) \in \llbracket c \rrbracket \}$ it is the opposite relation of $\llbracket c \rrbracket$

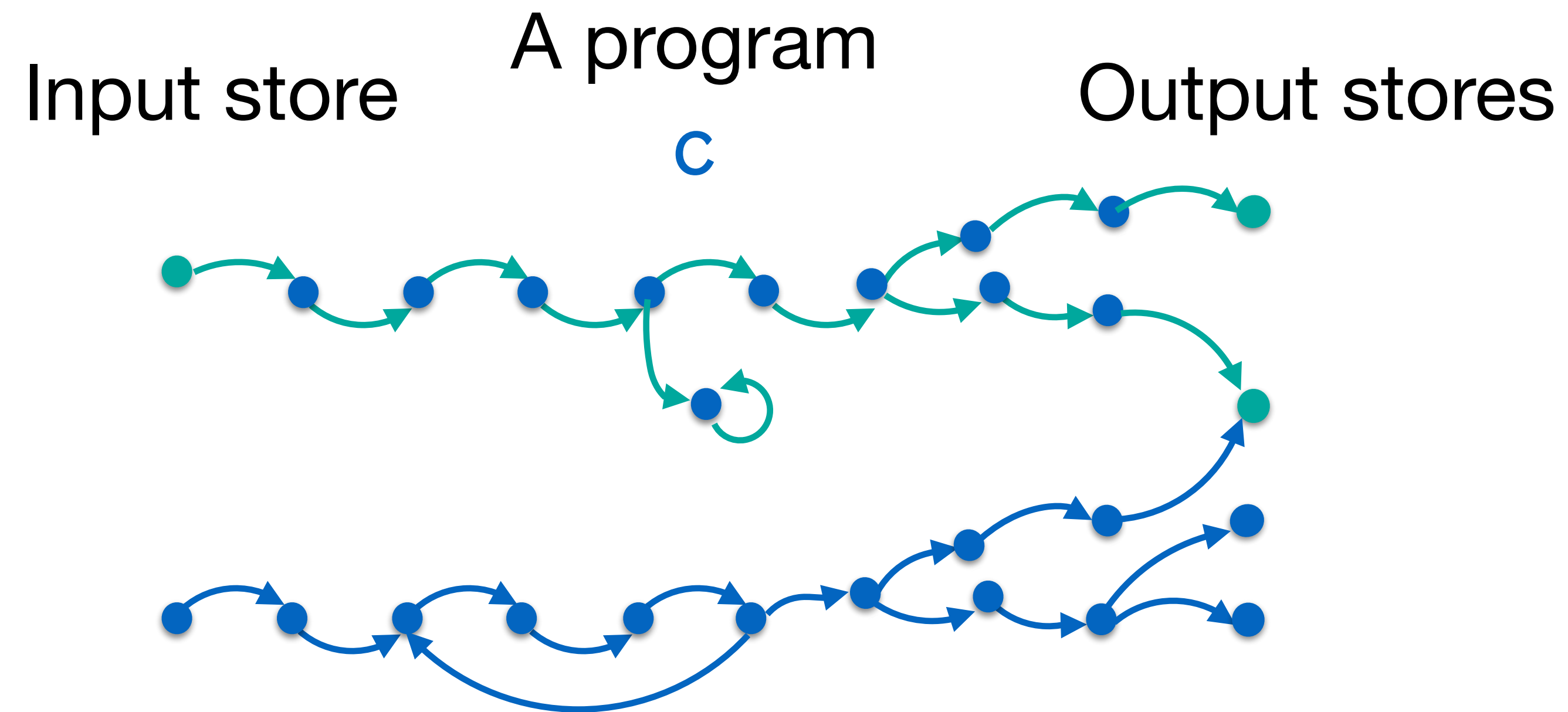
Therefore:

$\llbracket c \rrbracket \sigma = \{ \delta \mid (\sigma, \delta) \in \llbracket c \rrbracket \}$ and $\llbracket c \rrbracket^{\text{op}} \delta = \{ \sigma \mid (\sigma, \delta) \in \llbracket c \rrbracket \}$

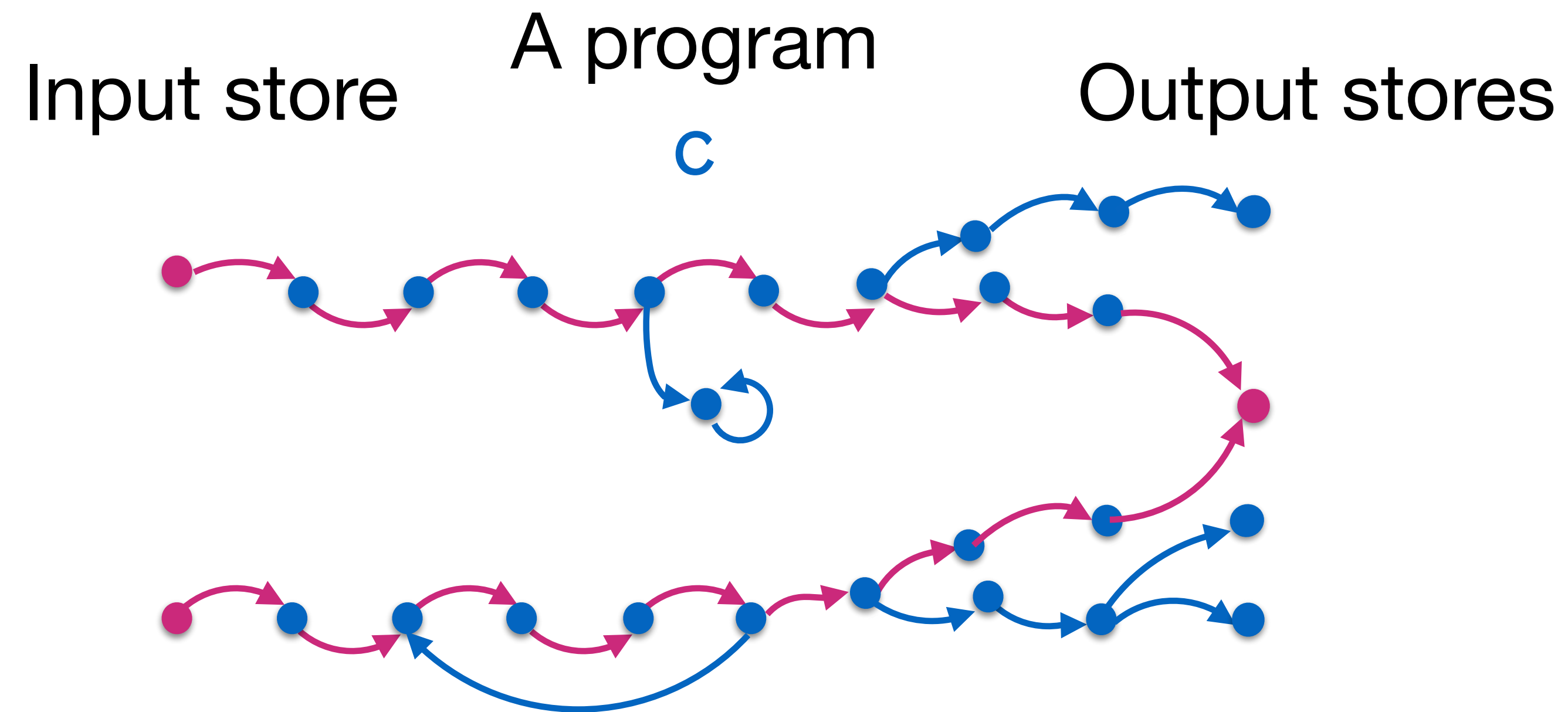
i.e.

$\delta \in \llbracket c \rrbracket \sigma$ if and only if $\sigma \in \llbracket c \rrbracket^{\text{op}} \delta$

Forward vs backward



Forward vs backward



Regular commands, backward

$$[[\text{skip}]]^{\text{op}} \triangleq \{ (\sigma, \sigma) \mid \sigma \in \Sigma \}$$

$$[[x := a]]^{\text{op}} \triangleq \{ (\sigma[[a]]\sigma/x, \sigma) \mid \sigma \in \Sigma \}$$

$$[[b?]]^{\text{op}} \triangleq \{ (\sigma, \sigma) \mid \sigma \models b \}$$

$$[[c_1; c_2]]^{\text{op}} \triangleq [[c_1]]^{\text{op}} \circ [[c_2]]^{\text{op}}$$

$$[[c_1 + c_2]]^{\text{op}} \triangleq [[c_1]]^{\text{op}} \cup [[c_2]]^{\text{op}}$$

$$[[c^\star]]^{\text{op}} \triangleq \bigcup_{k=0}^{\infty} ([c]^{\text{op}})^k$$

Hoare's weakest possible precondition

Given a program c and a correctness specification Q
what is the largest set of input states that have a "successful" computation?

$$wpp(c, Q) = \{\sigma \mid \llbracket c \rrbracket \sigma \cap Q \neq \emptyset\}$$

$$\llbracket c \rrbracket^{\text{op}} Q = wpp(c, Q)$$

wpp vs wlp

Given a program c and a correctness specification Q
what is the largest set of input states that have a "successful" computation?

$$wpp(c, Q) = \{\sigma \mid \llbracket c \rrbracket \sigma \cap Q \neq \emptyset\}$$

$$wlp(c, Q) = \{\sigma \mid \llbracket c \rrbracket \{\sigma\} \subseteq Q\}$$

Given a program c and a correctness specification Q
what is the largest set of input states whose computations are all "successful"?

Questions

Question 1

Let $c \triangleq (z := x) + (z := y)$

and let $P \triangleq (x = y = 0)$

- What is $\llbracket c \rrbracket P$?
- Give an example of over-approximation
- Give an example of under-approximation
- What is $wlp(c, (z = 0))$?
- What is $wpp(c, (z = 0))$?

$$(x = y = z = 0)$$

$$(x = y = 0)$$

$$(x = 0 \vee y = 0)$$

Question 2

Let $c \triangleq \text{if } x < y \text{ then } x := y \text{ else (while true do skip)}$

and let $Q \triangleq (x = y = 0)$

- What is $wlp(c, Q)$? $(x \geq y \vee y = 0)$
- What is $wpp(c, Q)$? $(x < y \wedge y = 0)$

Question 3

$$\llbracket c \rrbracket^{\text{op}} Q = wpp(c, Q)$$

$$\llbracket c \rrbracket(wpp(c, Q)) \text{ ? } Q$$

none!

some states in Q can be unreachable

$$wpp(c, \llbracket c \rrbracket P) \text{ ? } P$$

none!

some states in P can non-terminate